TICHU CYPRUS

Project Plan & Technical Specification

A modern mobile Tichu card game & scoring companion for the Cyprus community

Platform: iOS + Android (Flutter)
Architecture: Flutter/Flame + Node.js WebSocket + Supabase
Phase A (Estimation Companion): ~1-2 weeks
Phase B (Tichu Online): ~10-14 weeks additional
Estimated cost: ~$5/month

April 2026

Project Brief

Tichu Cyprus is a mobile card game app (iOS + Android) for the Cyprus and broader Mediterranean Tichu community. It starts with a scoring companion for IRL trick-prediction card games (Estimation / Oh Hell), then expands to include real-time online Tichu with friends or matchmaking, ranked seasons, and premium card animations. The primary goal is a passion project that grows into the go-to card game app for the Cyprus community.

Target audience: Cyprus card game community, Mediterranean card game players

Languages: Greek (default for Cyprus locale) + English

Design direction: Premium modern with Balatro-inspired aesthetic — dark rich backgrounds, glass-morphic UI panels, gold/amber accents.

Build Strategy

The app ships in two phases. Phase A (Estimation companion) gets the app into friends' hands within 1-2 weeks. Phase B (Tichu online) builds on the proven infrastructure. Nothing in Phase A is throwaway — auth, rooms, friends, themes, and CI/CD all carry forward.

Phase A: Estimation Scoring Companion

A real-time scoring companion for IRL trick-prediction card games. Players play with real cards; the app tracks predictions, validates scores via peer approval, and shows live standings. Replaces the person stuck with Excel.

Game Rules Encoded

- Players: 2-4
- Card count per round goes up then back down: 1, 2, 3 ... N ... 3, 2, 1
- With 4 players and a 52-card deck: max N = 13, total = 25 rounds
- Each round: every player predicts how many tricks they will win (0 to N)
- Predictions CAN sum to N (no restriction on the last bidder)
- Scoring: +1 point per trick won. +10 bonus if tricks won equals prediction exactly.
- Missed prediction: only trick points, no bonus
- Game ends after all rounds complete. Highest total score wins.

Core Loop: Predict → Play → Submit → Validate

Every round follows 4 steps. Each player is on their own phone in a shared Supabase Realtime session.

Step 1: Predict

App shows “Round 7 — 7 cards each.” Each player privately enters their prediction (stepper: 0 to N). Predictions are hidden until all players lock in. Then all revealed simultaneously — no peeking. This prevents anchoring bias from seeing others’ predictions.

Step 2: Play IRL

Players put phones down and play the round with real cards. The app shows everyone’s predictions as a reference. Dealer indicator visible. When the round is done, players pick up phones again.

Step 3: Submit Results

Each player taps how many tricks they actually won (stepper: 0 to N). The app runs a sanity check: total tricks submitted must equal N (the number of cards in that round). If they don’t add up, someone made a mistake — the app flags the mismatch and blocks advancement until corrected.

Step 4: Peer Validation

All submissions are shown to everyone: name, predicted, actual, calculated score. Each player taps “Confirm” or “Dispute.” Once 3 of 4 confirm (or all players in smaller games), scores are locked and the round advances. If disputed: the relevant player edits their entry and re-submits. Everyone re-validates.

This peer-validation system replaces the single scorekeeper. Instead of trusting one person with Excel, everyone verifies everyone. No more asking “who’s winning?” — everyone sees the live standings on their own phone.

Estimation Database Schema

3 new tables alongside the shared players table. All synced via Supabase Realtime.

estimation_games

Column	Type	Notes
id	uuid PK	Generated on room creation
room_code	text	4-character code for joining
player_count	int (2-4)	Number of players in this game
max_cards	int	Auto-calculated: floor(52 / player_count)
current_round	int	Which round is active (1-based)
total_rounds	int	Auto-calculated: 2 * max_cards - 1
status	enum: waiting | active | finished	Game lifecycle
phase	enum: predicting | playing | submitting | validating	Current step in the round loop
dealer_seat	int	Rotates each round (0 to player_count-1)
created_at	timestamptz	When game was created

estimation_players

Column	Type	Notes
id	uuid PK	
game_id	uuid FK → estimation_games	Which game session
player_id	uuid FK → players	Links to shared players table
seat	int (0-3)	Seating position

Column	Type	Notes
total_score	int DEFAULT 0	Running total, updated each round
joined_at	timestamptz	When player joined the room

estimation_rounds

One row per player per round. Stores prediction, actual result, and calculated score.

Column	Type	Notes
id	uuid PK	
game_id	uuid FK → estimation_games	Parent game
player_id	uuid FK → players	Which player
round_number	int	Which round (1-based)
cards_this_round	int	Number of cards dealt this round
prediction	int (nullable)	Null until player locks in prediction
actual_tricks	int (nullable)	Null until player submits result
score	int (nullable)	Auto-calculated: tricks + (prediction === actual ? 10 : 0)
validated	bool DEFAULT false	True once peer validation passes

Round Progression Logic

The card count per round follows the up-then-down pattern. With 4 players (max 13 cards):

Rounds 1-13: cards = round_number (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13)

Rounds 14-25: cards = 2 * max_cards - round_number (12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1)

Formula: cards_this_round = round_number <= max_cards ? round_number : 2 * max_cards - round_number

Player count variations:

- 2 players: max 26 cards each, 51 total rounds
- 3 players: max 17 cards each, 33 total rounds
- 4 players: max 13 cards each, 25 total rounds

Technical Approach

The companion runs entirely on Supabase Realtime — no WebSocket game server needed. This is a shared-state problem, not a game-logic problem.

- Each game session is a row in estimation_games with a phase field
- Phase transitions (predicting → playing → submitting → validating) trigger Supabase Realtime broadcasts
- Each client subscribes to game state changes and renders the appropriate UI for the current phase
- Predictions are written individually; a Supabase trigger checks if all predictions are in, then advances phase
- Trick count sanity check: app sums all actual_tricks and compares to cards_this_round
- Validation requires 3/4 confirms (or unanimous in 2-3 player games) before phase advances
- No hidden information after prediction lock-in — unlike Tichu, there's no card-peeking risk

Smart Features

- Trick count sanity check — totals must equal N, blocks advancement if wrong
- Auto-calculate scores — no mental math required
- Live leaderboard updated after every round with animated rank changes
- Score graph — line chart showing score progression over rounds, one line per player
- Prediction accuracy tracker per player (hit rate across the game)
- Dealer rotation indicator — auto-advances each round
- Game summary screen: winner, stats, fun awards ("Best Predictor," "Biggest Upset")
- Game history stored in Supabase for bragging rights

Phase A Screens

All portrait mode. Phones sit on or near the card table.

Screen	Contents
Create / Join	Score Companion button on Play tab. Create room (get code) or Join room (enter code). Player slots filling in.
Prediction Phase	Round number + card count header. Stepper (0 to N). Lock-in button. Waiting indicator for others. Simultaneous reveal.
Play Phase (Passive)	All predictions visible as reference table. Dealer badge. Round progress (e.g., Round 7 of 25). "Done Playing" button.
Submit Results	Stepper for tricks won (0 to N). Sanity check total displayed. Submit button. Mismatch warning if totals wrong.
Validation	All 4 results: name, predicted, actual, score. Total tricks check. Confirm / Dispute buttons. Progress: 2/4 confirmed.
Live Scoreboard	Always accessible via tab/button. Ranked player list. Score graph. Prediction accuracy. Round-by-round breakdown.

Screen	Contents
Game Over	Final standings. Winner highlight. Stats: total points, prediction accuracy, best round. Fun awards. Play Again button.

Phase A Milestones

3 milestones over ~1-2 weeks. Each produces something testable.

#	Milestone	Duration	Key Tasks	Deliverable
A1	Scaffold + Auth + Rooms	~3-4 days	Flutter project with Riverpod + Supabase SDK. Auth (Google/Apple/email). Username picker. Room code system via Supabase Realtime. Estimation DB tables. Play tab with Score Companion button.	Players sign in, create/join a room, see each other.
A2	Core Companion Flow	~4-5 days	Full predict → play → submit → validate loop. Prediction lock-in with simultaneous reveal. Trick count sanity check. Peer validation (3/4 confirm). Auto-scoring. Round progression (up then down). Game over detection.	Complete working companion for a full 25-round game.
A3	Polish + Scoreboard	~2-3 days	Live leaderboard with animated rank changes. Score graph (line chart). Prediction accuracy tracker. Dealer rotation indicator. Game summary with fun awards. Premium dark theme. Game history in Supabase.	Polished companion ready for TestFlight / internal testing.

Everything built in Phase A carries forward to Phase B (Tichu). Auth, player profiles, room codes, friends list, online status, push notifications, bottom tab navigation, premium dark theme, CI/CD, Supabase setup — all reused directly. Phase B only adds: Flame game engine, WebSocket game server, card rendering, game logic, animations, landscape mode, and matchmaking.

Phase B: Tichu Online

The full real-time online Tichu card game. Builds on Phase A's proven infrastructure.

Tichu Feature Map (MoSCoW)

Must Have (MVP)

- Complete Tichu game engine (all rules, 56-card deck, special cards, scoring)
- Real-time 4-player multiplayer via WebSocket with authoritative server
- Private rooms (create room, share 4-character code, join)
- Auto-matchmaking with ELO-based skill pairing
- Card animations (deal, play to table, trick capture)
- Auto-scoring engine with end-of-round breakdown and dispute window
- Always-visible scoring reference card (expandable cheat sheet)
- In-game point floaters (+5, +10 on high-value card captures)

Should Have (Post-MVP)

- Ranked seasons with ELO leaderboard and seasonal resets
- Player stats and match history with round-by-round breakdown
- Reconnection handling (dropped connections rejoin mid-game)
- Spectator mode
- In-game quick emojis / reactions
- Game replay / review
- Kafeneio mode — nearby player discovery via opt-in location sharing
- Animated cinematic scoring tally (Balatro-style card flip sequence)
- Interactive tutorial for new players

Could Have

- In-game text chat
- Custom card backs / table themes
- Achievement badges
- Sound effects and haptics
- Post-game detailed stats and shareable game summary card

Won't Have (Out of Scope)

- AI bots, tournaments/leagues, clubs/teams, in-app purchases, voice chat

Technical Architecture

Component	Technology	Rationale
Mobile client	Flutter + Dart 3.x	Best animation ceiling for card games. Impeller engine, no bridge bottleneck.
Game rendering	Flame engine	Purpose-built 2D game engine for Flutter. Particles, sprites, physics.
State management	Riverpod	Compile-time safe, provider scoping. Complex game state.
Game server	Node.js + TypeScript	Authoritative game logic. Validates all moves, prevents cheating.
Real-time transport	Raw WebSockets (ws)	Full control over game protocol. Sub-100ms round trip.
Database	Supabase (PostgreSQL)	Profiles, stats, history, friends. Auto REST API + RLS.
Authentication	Supabase Auth	Google + Apple sign-in. Already proven in Phase A.
Push notifications	Firebase Cloud Messaging	Industry standard, free, universal.
Hosting (server)	Railway	Simple deploy, WebSocket support, ~$5/month.
CI/CD	GitHub Actions + Fastlane	Proven in Phase A. App Store / Play Store uploads.
Error tracking	Sentry	Flutter + Node.js SDKs. Free tier.

Architecture principle: During a live Tichu game, the database is never touched. The game server holds the entire game state in memory for speed. Only when a round or game finishes does it write results to Supabase. This separation gives sub-100ms response times during gameplay.

Tichu Database Schema

7 tables for Tichu data, alongside the shared players table and 3 Estimation tables.

games

Column	Type	Notes
id	uuid PK	Generated by game server
room_code	text	Null for matchmade games
game_type	enum: casual | ranked	Affects ELO calculation
status	enum: waiting | active | finished | abandoned	Lifecycle state
team_a_score / team_b_score	int DEFAULT 0	Running totals across rounds
winning_team	enum: a | b | null	Set when game ends
target_score	int DEFAULT 1000	Game ends when a team reaches this
started_at / ended_at	timestamptz	Game timing

game_players

Column	Type	Notes
id	uuid PK	
game_id	uuid FK → games	Which game
player_id	uuid FK → players	Which player
seat	enum: north | south | east | west	Table position
team	enum: a | b	North+South = A, East+West = B
elo_before	int	Snapshot for history display

rounds

Column	Type	Notes
id	uuid PK	
game_id	uuid FK → games	Parent game
round_number	int	Sequential (1, 2, 3...)
team_a_card_points / team_b_card_points	int	Points from card values

Column	Type	Notes
team_a_total / team_b_total	int	Cards + bonuses
one_two_finish	enum: team_a | team_b | null	Both partners out 1st+2nd
finish_order	text[]	Array of 4 seats in order

tichu_calls

Column	Type	Notes
id	uuid PK	
round_id	uuid FK → rounds	Which round
player_id	uuid FK → players	Who called
call_type	enum: tichu | grand_tichu	+/-100 or +/-200
successful	bool	Did they go out first?

round_logs

Complete replay data stored as JSON. Write once, read for replays.

Column	Type	Notes
id	uuid PK	
round_id	uuid FK → rounds	Which round
dealt_hands	jsonb	4 hands, 14 cards each
exchanges	jsonb	Who gave what to whom
tricks	jsonb[]	Ordered play-by-play

friendships

Column	Type	Notes
id	uuid PK	
requester_id / addressee_id	uuid FK → players	Friend pair
status	enum: pending | accepted | blocked	Lifecycle
created_at	timestamptz	When request sent

elo_history

Column	Type	Notes
id	uuid PK	
player_id	uuid FK → players	Whose rating changed
game_id	uuid FK → games	Which game
elo_before / elo_after	int	Rating change
created_at	timestamptz	When change occurred

WebSocket Message Protocol

Every message: { type, payload, seq }. Server is authoritative. Clients send actions, server broadcasts validated state.

Client → Server (11 message types)

Type	Payload	Description
room:create	{ gameType }	Create a private room
room:join	{ roomCode }	Join existing room
queue:join	{ }	Enter matchmaking
game:call_grand_tichu	{ }	After seeing first 8 cards
game:pass_grand_tichu	{ }	Decline Grand Tichu
game:call_tichu	{ }	Before first card played
game:exchange_cards	{ left, partner, right }	Pass 1 card to each player
game:play_cards	{ cards[], phoenixValue?, wish? }	Play a combination
game:pass	{ }	Pass on current trick
game:give_dragon	{ to: 'left' | 'right' }	Gift Dragon trick
game:confirm_score	{ }	Accept round scoring

Server → Client(s) (15 message types)

Type	Payload	Description
room:created	{ roomCode, gameId }	Room ready
room:player_joined	{ seat, player }	Player sat down
game:deal_first_eight	{ cards[8] }	Private: only to this player
game:deal_remaining_six	{ cards[6] }	After GT window
game:grand_tichu_called	{ seat }	Someone called GT
game:tichu_called	{ seat }	Someone called Tichu
game:exchange_complete	{ received[] }	Cards received from exchange
game:wish_made	{ rank }	Mahjong wish broadcast
game:cards_played	{ seat, cards, comboType, cardsRemaining }	Cards played to table
game:player_passed	{ seat }	Player passed
game:trick_won	{ winningSeat, points, dragonGiftRequired }	Trick resolved

Type	Payload	Description
game:wish_fulfilled	{ }	Wished rank played
game:player_finished	{ seat, position }	Player went out
game:round_result	{ full scoring breakdown }	End-of-round
game:game_over	{ winningTeam, finalScore, eloChanges, xpAwarded }	Game complete

Game phase state machine: DEAL_FIRST_8 → GRAND_TICHU_WINDOW → DEAL_REMAINING_6 → EXCHANGE → PLAY → ROUND_SCORING → NEXT_ROUND or GAME_OVER

Tichu Scoring System UX

Four layers of scoring UX that reinforce each other:

Layer 1: Always-On Reference Card

- Card points: 5s = 5, 10s = 10, Kings = 10. Dragon = +25, Phoenix = -25
- Tichu: +100 success / -100 fail. Grand Tichu: +200 / -200
- 1-2 finish: flat 200, skip counting

Layer 2: Live In-Game Awareness

- Running point counter per team. Point floaters on high-value captures. Tichu badge glows.

Layer 3: End-of-Round Cinematic Breakdown

1. Won cards fly to team pile, flip face-up, tally with click-click-click sound
2. Last player's cards transferred to opponents (animated with tooltip)
3. Tichu/Grand Tichu bonuses slam onto score
4. 15-second dispute/review window

Layer 4: New Player Tutorial

- First-launch walkthrough, interactive sample round, progressive disclosure

Tichu Screen Inventory

14 unique screens. Portrait for lobby, landscape for gameplay.

Bottom Tab Bar (Portrait)

Tab	Contents
Play	Quick Match, Create Room, Join Room, Score Companion, online count, recent matches
Friends	Friends list (online first), pending requests, search/add, invite to room
Leaderboard	Toggle: global / friends / Cyprus. Top 50 by ELO.
Profile	Avatar, rank badge, level + XP, 6 stat cards, ELO graph, match history

Game Screen (Landscape, Full-Screen Immersive)

4-seat card table layout. Your hand as fan at bottom. Partner top center. Opponents left/right. Center play area. Team scores in corners. Turn indicator.

Overlay states: Grand Tichu decision, card exchange, Dragon gift choice, Mahjong wish selector, round scoring breakdown, game over.

Infrastructure & DevOps

Environments

- Local: Flutter emulator + Node.js game server + Supabase local (Docker)
- Production: Railway (game server) + Supabase cloud + App Store / Play Store

CI/CD (GitHub Actions)

- Game server: push to main → Lint → Type check → Test → Deploy to Railway
- Flutter app: version tag → Analyze → Test → Build APK + IPA → Upload via Fastlane

Monitoring

- Error tracking: Sentry (Flutter + Node.js). Uptime: UptimeRobot. Logging: Railway built-in. Analytics: PostHog.

Monorepo Structure

- app/ — Flutter mobile app (lib/game, lib/screens, lib/providers, lib/services)
- server/ — Node.js game server (src/game, src/ws, src/matchmaking)
- supabase/ — Migrations, seed data, Edge Functions
- shared/ — JSON schemas for protocols
- docs/ — ADRs, wireframes, game rules reference

Monthly Cost (MVP)

Service	Cost
Supabase (free tier)	$0
Railway (starter)	~$5
FCM + Sentry + PostHog	$0
Total	~$5/month

Security Plan

Estimation companion: minimal risk (no hidden information after prediction lock-in). Tichu: authoritative server prevents cheating by architecture.

Threat	Risk	Mitigation
Card peeking (Tichu)	High	SOLVED: Server only sends each player their own hand.
ELO manipulation	High	MITIGATED: Flag colluding accounts. Min games threshold.
Invalid moves (Tichu)	Medium	SOLVED: Server validates every play.
Score inflation (Estimation)	Medium	SOLVED: Peer validation (3/4 must confirm).
Replay attacks	Medium	SOLVED: Sequence numbers on all WS messages.

Complete Execution Plan

Two phases, 11 milestones total. Phase A ships in ~1-2 weeks. Phase B adds ~10-14 weeks.

Phase A: Estimation Companion (~1-2 weeks)

#	Milestone	Duration	Deliverable
A1	Scaffold + Auth + Rooms	~3-4 days	Players sign in, create/join rooms, see each other.
A2	Core Companion Flow	~4-5 days	Full predict → play → submit → validate for 25 rounds.
A3	Polish + Scoreboard	~2-3 days	Live leaderboard, score graph, dark theme. Ship to testers.

Phase B: Tichu Online (~10-14 weeks)

#	Milestone	Duration	Deliverable
B1	Game Engine (Server)	~10-14 days	Complete Tichu logic, fully tested. Playable via raw WebSocket.
B2	Game Board UI	~10-12 days	Play full Tichu on two phones via room codes.
B3	Matchmaking + Friends	~5-7 days	Full multiplayer: friends + stranger matchmaking.
B4	Premium Visuals	~10-14 days	Dark theme, card animations, cinematic scoring, haptics.
B5	Progression + Community	~5-7 days	XP, ranked ELO, leaderboards, push notifications, i18n.
B6	Polish + Ship	~5-7 days	Edge cases, beta test, submit to App Store + Play Store.

Timeline Summary

Phase	Duration	Cumulative
Phase A: Estimation companion	~1-2 weeks	Week 2
Phase B1-B2: Tichu engine + game board	~3-4 weeks	Week 6
Phase B3-B5: Features + polish	~4-5 weeks	Week 11
Phase B6: Final polish + ship Tichu	~1-2 weeks	Week 13
Total (both modes live)	~12-15 weeks	

Post-Launch Roadmap

- v1.1 — Interactive Tichu tutorial for new players
- v1.2 — Kafeneio mode (nearby player discovery)
- v1.3 — Spectator mode + game replays
- v1.4 — Additional companion game modes (Barbu, Prefa, etc.)
- v2.0 — Tournaments + seasonal events

Phase A gets a working app into your friends' hands in under two weeks. Phase B builds on that foundation. By the time Tichu ships, your infrastructure is battle-tested, your app is already installed, and your community is already playing together.